



中国科学技术大学

文件操作与数据持久化

吴锋

Email: wufeng02@ustc.edu.cn



本章内容概述

- ❖ 文件操作与数据持久化
- ❖ 二进制文件与序列化
- ❖ 文件的定位与随机访问
- ❖ 文件的错误处理



本章内容概述

- ❖ 文件操作与数据持久化
- ❖ 二进制文件与序列化
- ❖ 文件的定位与随机访问
- ❖ 文件的错误处理



文件的基本概念

- ❖ 文件是一组相关数据的有序集合，信息最终是**以文件形式存储在永久存储设备中**（如硬盘、光盘、U盘等）
- ❖ **数据持久性**：内存（断电即失） vs 磁盘（永久保存）
- ❖ 操作系统及应用程序往往通过对文件的输入输出完成某项功能；操作系统将与主机相连的输入输出设备也视为文件，如键盘是输入文件，显示器是输出文件
- ❖ 文件按结构形式分为**文本文件**和**二进制文件**
 - ❧ 文本文件是全部由**字符**组成（用ASCII码表示）的具有行列结构的文件，又称为ASCII码文件。便于对字符进行处理，可以直接显示
 - ❧ 二进制文件是按数据在**内存**中的存储形式存储到文件中，一般不能直接显示
 - ❧ 文本文件的ASCII码也是二进制的，区别在于数值。例：int型整数5在文本文件中保存的是0x35，而在二进制文件中是0x00,0x00,0x00,0x05四个字节



文件的基本概念

❖ 文件按数据的物理存取方式分为**顺序文件**和**随机文件**

∞ **顺序文件**：数据在存储介质中的实际顺序与它们进入存储器的顺序一致；一切存储在顺序存储器（如磁带）上的文件，都只能是顺序文件（难以在中间定位）



A记录	B记录	C记录
-----	-----	-----

∞ **随机文件**：数据散列在存储介质中；硬盘上的文件，一般是随机文件（系统工具中的磁盘整理就是尽可能将分散随机存储的文件数据集中放置，以便于就近读写）



A记录		B记录	C记录	
-----	--	-----	-----	--



文件的基本概念

❖ 文件的读写方式分为顺序存取和随机存取

∞ 顺序存取：从头到尾读写文件所有数据，文件被看成一个字符流，称为流式文件

❖ 网络中所说的流是指一组有序数据序列，代表传输中的数字序列

❖ 向文件顺序写入/读出数据

∞ 打开文件并将文件指针置于文件的开头

∞ 将内存中的数据顺序写入文件/将数据顺序读入内存

∞ 随机存取：随机从文件内指定的位置读写数据

❖ 向文件随机写入/读出数据

∞ 打开文件并将文件定位指针置于待写入/读出的位置

∞ 将内存中的数据写入文件/将数据读入内存

∞ 顺序文件只能顺序存取，随机文件两种都可以



文件的基本概念

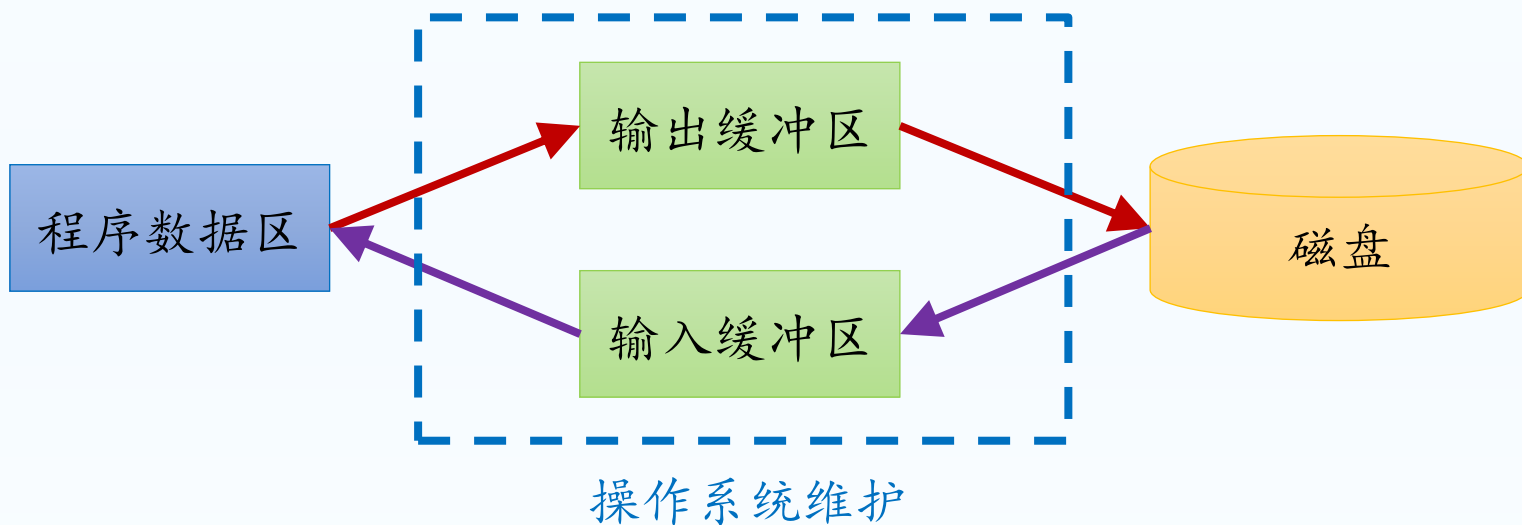
- ❖ 按存储的外部设备分为**磁盘文件**（普通文件）和**设备文件**
 - ❧ **磁盘文件**是指存放在磁盘或其它外部介质上的有序数据集
 - ❧ 操作系统把与设备间的输入输出等同于对磁盘文件的读写，称其为**设备文件**，包括与主机相连的各种外部设备，如显示器、键盘等
 - ❧ 通常把显示器定义为**标准输出文件(stdout)**，向stdout文件输出，即为在显示器上显示
 - ❧ 类似的，把键盘定义为**标准输入文件(stdin)**，从stdin文件读入，即为从键盘读入



文件的基本概念

❖ 缓冲文件系统和非缓冲文件系统

- ❧ 缓冲文件系统是指系统自动地在内存中为每一个正在使用的文件都开辟一个缓冲区
- ❧ 从内存向磁盘输出数据时，缓冲文件系统先将数据送到内存中的缓冲区，待缓冲区满时才将整个缓冲区的数据一起保存到磁盘文件中（或强制写入）
- ❧ 从磁盘向内存读入数据时，一般是从磁盘文件中一次读入一批数据到缓冲区，程序从缓冲区获取数据，缓冲区空时再次读入一批数据





文件的基本概念

❖ 缓冲文件系统和非缓冲文件系统

- ❧ 非缓冲文件系统是指系统不自动开辟缓冲区，而由使用文件的程序直接操作文件数据，自行开辟和维护缓冲区
- ❧ 现代操作系统和运行环境都采用缓冲文件系统
 - ❖ 写入时不要遗漏缓冲区内尚未存入的数据
 - ❖ 读取时可能不是“即时”输入的数据
- ❧ 常见现象：为什么 `printf()` 后屏幕可能没立即显示？写入文件后立即打开该文件却可能是空的？
- ❧ 核心思想：磁盘 I/O 极慢，CPU 极快；数据先堆积在内存缓冲区，暂缓写入磁盘（减少慢速的 I/O 次数）



文件类型与文件指针

- ❖ 操作系统读写文件时需要的信息
 - ❧ 文件当前的读写位置
 - ❧ 与文件对应的内存缓冲区地址
 - ❧ 文件的操作方式，等
- ❖ 以上信息都存放在“文件信息区”中，一个由系统定义的结构体类型变量，该结构体类型在stdio.h中定义，标识符是FILE，具体定义与编译系统有关
- ❖ C语言标准函数中，系统自动打开和关闭三个标准文件指针
 - ❧ 标准输入（默认绑定键盘）stdin
 - ❧ 标准输出（默认绑定显示器）stdout
 - ❧ 标准出错输出（默认绑定显示器，且无缓冲）stderr



标准输入输出命令行重定向

- ❖ 输入重定向（将`stdin` / 0 重定向，从文件读取内容代替键盘输入）：
 - ❧ 方法：`./my_program < input.txt`
 - ❧ 效果：`scanf()` 等会从 `input.txt` 文件中读取输入，无需从键盘输入
- ❖ 输出重定向（将`stdout` / 1 重定向，原本打印在屏幕的内容写入文件）
 - ❧ 方法：`./my_program > output.txt` （文件如果有内容会被清空）
 - ❧ 追加：`./my_program >> output.txt` （输出内容补充在文件末尾）
 - ❧ 效果：`printf()` 等输出结果会保存在 `output.txt` 文件中，屏幕无显示
- ❖ 错误重定向（将 `stderr` / 2 重定向，仅将错误信息写入文件）：
 - ❧ 方法：`./my_program 2> error.log`
- ❖ 所有输入输出都重定向到文件（上述重定向可以随意组合）：
 - ❧ 方法：`./my_program < input.txt > output.txt 2 > error.log`
- ❖ 一个程序的标准输出作为另一个程序的标准输入：`random | sum`



文件类型与文件指针

❖ FILE结构体示例 (stdio.h)

```
struct _iobuf {  
    int _file;           // 文件描述符  
    int _flag;           // 文件状态标志  
    char *_ptr;          // 指向当前读取/写入位置的指针  
    int _bufsiz;         // 缓冲区大小  
    int _cnt;            // 缓冲区中的字节数  
    int _charbuf;        // 用于存储单个字符的缓冲区  
    char *_base;         // 指向缓冲区起始位置的指针  
    char *_tmpfname;     // 临时文件名 (如果有)  
    ... ..              // 可能还会有其他字段  
};  
typedef struct _iobuf FILE;
```

❖ 缓冲文件系统中，关键的概念是“文件指针”，定义文件指针的形式：

```
FILE *fp1,*fp2;
```

❖ 通过文件指针可以找到存放该文件信息的结构变量，然后按结构变量提供的信息找到文件并进行操作



文件的打开

- ❖ 文件需要打开才能操作，打开文件的实质是在用户程序和操作系统间建立起联系，通过文件指针共享文件信息
- ❖ C语言使用标准库函数 `fopen()` 打开或新建文件，函数原型：
`FILE *fopen(const char*fname, const char*mode);`
 - ❧ `fname` 是将要访问的文件名，**路径必须有效**，否则可能找不到文件
 - ❧ `mode` 是文件模式，用以规定文件可以操作的方式，是一个**字符串**
 - ❧ `FILE *` 是函数返回类型，注意这里返回的是一个**文件指针**，文件的实质结构体变量由系统产生和维护



文件的绝对路径与相对路径

- ❖ **绝对路径**：从根目录（如 `C:\` 或 `/`）开始的完整路径
 - ∞ Windows系统（分隔符 `\`）：`C:\Users\Admin\Desktop\data.txt`
 - ∞ Linux系统（分隔符 `/`）：`/home/user/project/data.txt`
- ❖ **相对路径**：从当前（可执行程序）所在目录开始的路径
 - ∞ 当前目录（可以省略）：`./data.txt` 或 `data.txt`
 - ∞ 上级目录（可以叠加）：`../config.ini` , `../../config.ini`
- ❖ **注意**：在C语言字符串中，需要用 `"\\"` 表示 `\`
 - ∞ 例如：`fopen("C:\\Users\\Admin\\Desktop\\data.txt", "r")`



文件的操作模式

❖ 文件操作模式 mode

文件操作模式	意义
"r"	以只读方式打开文本文件，该文件应该存在，不存在则报错
"w"	以只写方式建立文本文件，若文件存在则清空文件内容；若文件不存在则建立该文件
"a"	以追加的方式打开文本文件。若文件不存在，则会建立该文件；如果文件存在，则从文件尾开始写（不会清空文件）

∞ rb、wb、ab 与 r、w、a 相似，但对象是二进制文件

∞ r+、w+、a+ 以读写的操作模式打开文本文件，可切换读写模式

∞ rb+、wb+、ab+ 以读写的操作模式打开二进制文件，可切换读写模式



文件的操作模式（续）

❖ 读写模式（+）的作用

基础模式	读写模式（+）	核心区别与应用场景
"r" (只读) 文件必须存在	"r+" (读写) 文件必须存在	多了【修改】的能力。 r: 只能读，不能写。 r+: 可以修改文件中间的任意字节。 场景：修改游戏存档、更新数据库记录。
"w" (只写) 清空/创建	"w+" (读写) 清空/创建	多了【回看】能力。 会清空原有内容，只能读取刚刚写入的数据。 场景：临时文件写完，立刻需要读出来处理。
"a" (追加) 创建/尾部	"a+" (读写) 创建/尾部	多了【读取】能力。 a: 只能向后写。 a+: 可以读取整个文件的历史记录，但写入时强制在末尾添加内容。 场景：聊天软件 (读取历史消息 + 发送新消息)。

思考：修改文件中间的某个字符（原地修改），如果使用 w+ 文件会怎么样？



文件的打开 (例)

```
FILE *fp1, *fp2, *fp3;  
char filename[]="file3.dat";  
  
// 以文本只读方式打开file1  
if (!(fp1=fopen("file1", "r"))) {  
    printf("Cannot Open This File!\n");  
    exit(0); // 退出程序  
}
```

注意路径中的反斜线
需要用字符转换

```
// 以二进制读写方式打开FILE2.TXT  
fp2=fopen("C:\\HOME\\FILE2.TXT", "rb+");  
// 以二进制读写方式打开file3.dat  
fp3=fopen(filename, "a+b");
```

顺序并不严格



文件的关闭

- ❖ 打开文件后，系统会分配文件的缓冲区，文件使用完后应及时关闭文件以释放缓冲区
- ❖ C语言使用库函数 `fclose()` 关闭文件，原型：
`int fclose(FILE *stream);`
 - ⌘ `stream`是打开文件时获得的文件指针（流指针）
 - ⌘ 文件正常关闭后返回值为0，否则返回 `EOF` 即-1
 - ⌘ 示例：`fclose(fp);`
- ❖ 关闭文件还有一个重要作用是操作系统此时将文件缓冲区的剩余内容写入文件，正常关闭文件才能确保文件内容的正确



文件的关闭（例）

// 典型代码范式

```
FILE *fp = fopen("demo.txt", "w");
```

```
if (fp) {
```

```
    fprintf(fp, "Hello Persistence");
```

// 此时数据可能还在缓冲区

```
fclose(fp);
```

// 现在数据一定在磁盘上了

```
fprintf(fp, "Hello Again"); // 错误：关闭文件后不能再操作文件
```

```
fp = NULL; // 好习惯：关闭文件后将文件指针置为 NULL
```

```
}
```



文件指针复用

- ❖ `freopen()` 用于新打开一个文件，直接关联到某个已经打开的文件指针，这样可以复用文件指针，原型定义在头文件 `stdio.h`

`FILE* freopen(char* fname, char* mode, FILE* stream);`

- ❖ `freopen()` 会自动关闭原先已经打开的文件，如果文件指针并没有指向已经打开的文件，则 `freopen()` 等同于 `fopen()`

```
freopen("output.txt", "w", stdout); // stdout 重定向到 output.txt 文件  
printf("hello"); // 输出 hello 到 output.txt 文件
```

```
freopen("input.txt", "r", stdin); // stdin 重定向到 input.txt 文件  
scanf("%d", & i2); // 从 input.txt 文件读取一个整数
```

- ❖ 某些系统允许使用 `freopen()`，改变文件的打开模式。这时，`freopen()` 的第一个参数应该是 `NULL`

```
freopen(NULL, "wb", stdout); //将stdout的打开模式从w改成了wb
```



文件的读写

❖ 文本的文件读写函数

- ❧ 格式化读写函数 `fscanf()` 和 `fprintf()`
- ❧ 字符串读写函数 `fgets()` 和 `fputs()`

❖ 二进制文件的读写函数

- ❧ 数据块读写函数 `fread()` 和 `fwrite()`

❖ 字节（字符）读写函数

- ❧ 函数 `fgetc()` 和 `fputc()`，同时适用于文本文件和二进制文件的读写
- ❧ 注意：用文本或二进制方式打开文件后，并不限制是用文本还是二进制形式读写数据，但结果会有差别（写值或写ASCII码，读换行符）



文本文件的读写

❖ 格式化（文本形式）读写函数

`int fscanf(FILE *stream, const char*format,...);`

☞ 返回值：成功读取的项目数量；若出现错误或到达文件末尾，返回 `EOF`。

`int fprintf(FILE *stream, const char*format,...);`

☞ 返回值：成功写入的字符数；若出现错误，返回负值。

❖ 用法类似`scanf()`与`printf()`，但增加文件指针作为首个形参

```
// fscanf(stdin, ...) 等效于 scanf, fprintf(stdout, ...) 等效于 printf
fscanf(fp, "%d%f", &i, &f);    //从指针fp所指的文件中读取一个整型
                                //数据到整型变量i中,再读取一个浮
                                //点型数据到浮点型变量f中
fprintf(fp, "%d", i);           //将整型变量i的值存储到fp所指文件中
```



文本文件的读写

❖ 字符串读写函数

`char *fgets(char *str, int n, FILE *stream);`

- ❧ `str`: 存储读取数据的缓冲区，必须确保足够大
- ❧ `n`: 要读取的最大字符数（包括终止符 `\0`），一般为 `sizeof(str)`
- ❧ 返回值: 成功读取一行，返回 `str`；若遇到文件结束或错误，则返回 `NULL`

`int fputs(const char *str, FILE *stream);`

- ❧ 返回值: 成功写入字符串，返回非负值；若出现错误，返回 `EOF`。

❖ 示例（通常用于逐行读取或者写入）

```
char buffer[100];  
while (fgets(buffer, sizeof(buffer), fp) != NULL) {  
    printf("%s", buffer); // 输出读取的行  
}  
fputs("Hello, World!\n", fp); // 写入字符串
```



文本文件的读写

❖ 用 `fgets()` 实现 `fscanf()` (按行读取并格式化解析)

```
fscanf(fp, "%d %s", & id, name); // fscanf 函数实现

// 用 fgets 实现
if (fgets(buffer, sizeof(buffer), fp) != NULL) {
    sscanf(buffer, "%d %s", & id, name); // 解析行数据
}
```

❖ 用 `fputs()` 实现 `fprintf()` (格式化后按行写入)

```
fprintf(fp, "%d %s\n", id, name); // fprintf 函数实现

// 用 fputs 实现
sprintf(buffer, "%d %s\n", id, name); // 格式化到缓冲区
fputs(buffer, fp); // 写入字符串
```




二进制文件的读写

❖ 数据块（二进制形式）的读写函数

`size_t fread(void *buf, size_t size, size_t n, FILE *stream);`

`size_t fwrite(const void *buf, size_t size, size_t n, FILE *stream);`

∞ `size_t` 是 `stdio.h` 中定义的类型 (`typedef unsigned int size_t;`)

∞ `buf` 为输入/输出在内存中存放的首地址

∞ `size` 为读/写的字节数，即数据块的大小

∞ `n` 为输入/输出的数据（块）项个数

∞ `stream` 为文件指针

∞ 返回值：成功读取/写入的元素数量



二进制文件的读写

❖ 数据块（二进制形式）的读写函数

∞ 调用形式

`fread(buf, size, n, fp);` // 从fp所指文件读入n个大小为size的数据块，存入buf所指内存，返回读取的数据项个数

`fwrite(buf, size, n, fp);` // 从buf所指数据区取size*n个字节写入fp所指文件，返回写到文件中的数据项个数

∞ 示例（用于结构变量的读写）

```
fread(&stu[i], sizeof(struct student), 1, fp);  
fwrite(&stu[i], sizeof(struct student), 1, fp);
```

∞ 示例（用于数组的读写）

```
fread(stu, sizeof(struct student), 3, fp);  
fwrite(stu, sizeof(struct student), 3, fp);
```



文件的字节读写

❖ 字节读写函数

`int fgetc(FILE *stream);`

∞ 返回值：读取一个字符，返回其ASCII 码；遇到文件结尾或错误，返回 **EOF**

∞ `fgetc(stdin)` 等效于 `getchar()`

`int fputc(int c, FILE *stream);`

∞ 返回值：成功写入字符，返回字符；若出现错误，返回 **EOF**

∞ `fputc(c, stdout)` 等效于 `putchar(c)`

❖ 例：

```
fp = fopen("test1.txt", "r");  
while ((ch = fgetc(fp)) != EOF) { .... }
```

```
fp = fopen("test1.bin", "rb");  
while ((ch = fgetc(fp)), !feof(fp)) { .... }
```



文件的字节读写

- ❖ **EOF** 是一个系统宏 (通常是 -1), 由系统内核返回, 表示 “到达文件末尾” (End Of File), 而不是文件中的一个字符 (不存储在文件中)
- ❖ 注意: 接收变量的类型
 - ∞ `fgetc()` 返回的是 `int`, 而不是 `char`
 - ∞ 如果用 `char` 接收, 可能无法区分 `0xFF` (?) 和 `EOF` (-1)
- ❖ 标准写法: 先读, 判断不等于 `EOF`, 再处理

```
// 典型错误写法
char c;
// char 范围可能覆盖不了 EOF
// 可能死循环或提前退出
while ((c = fgetc(fp)) != EOF)
{
    printf("%c", c);
}
```

```
// 标准正确写法
int c; // 必须用 int 来接收

while ((c = fgetc(fp)) != EOF)
{
    printf("%c", c);
}
```



文件的字节读写（例一）

❖ 用 `fgetc()` 实现 `fgets()` (模拟读取字符串)

```
char* my_fgets(char* str, int n, FILE* stream) {
    int c;
    char* cs = str;
    // 循环读取，直到达到n-1个字符或遇到换行符或EOF
    while (--n > 0 && (c = fgetc(stream)) != EOF) {
        if ((*cs++ = c) == '\n') { // 如果读取到换行符，停止
            break;
        }
    }
    *cs = '\0'; // 结尾补\0
    // 如果未读取任何内容就遇到EOF，返回NULL
    if (cs == str && c == EOF) {
        return NULL;
    }
    return str;
} // 读取直到换行符 \n 或读取到n-1个字符，并确保字符串以 \0 结尾
```



文件的字节读写（例二）

❖ 用 `fputc()` 实现 `fputs()` (模拟写入字符串)

```
int my_fputs(const char* str, FILE* stream) {  
    while (*str != '\0') {  
        if (fputc(*str, stream) == EOF) {  
            return EOF; // 写入失败  
        }  
        str++;  
    }  
    return 0; // 成功  
} // 将字符串写入文件，不包含 \0，不自动添加换行符
```



文件的字节读写（例三）

❖ 用 `fgetc()` 实现 `fread()` (模拟读取二进制数据块)

```
size_t my_fread(void *ptr, size_t size, size_t nmemb, FILE *stream)
{
    char *p = (char *)ptr;
    size_t total_bytes = size * nmemb;
    size_t read_bytes = 0;
    for (size_t i = 0; i < total_bytes; i++) {
        int c = fgetc(stream);
        if (c == EOF) {
            break; // 读取到文件末尾或出错
        }
        p[i] = (char)c;
        read_bytes++;
    }
    return read_bytes / size; // 返回成功读取的元素个数
}
```



文件的字节读写（例四）

❖ 用 `fputc()` 实现 `fwrite()` (模拟写入二进制数据块)

```
size_t my_fwrite(const void *ptr, size_t size, size_t nmemb, FILE
*stream) {
    const char *p = (const char *)ptr;
    size_t total_bytes = size * nmemb;
    size_t written_bytes = 0;

    for (size_t i = 0; i < total_bytes; i++) {
        if (fputc(p[i], stream) == EOF) {
            break; // 写入出错
        }
        written_bytes++;
    }
    return written_bytes / size; // 返回成功写入的元素个数
}
```




本章内容概述

- ❖ 文件操作与数据持久化
- ❖ 二进制文件与序列化
- ❖ 文件的定位与随机访问
- ❖ 文件的错误处理



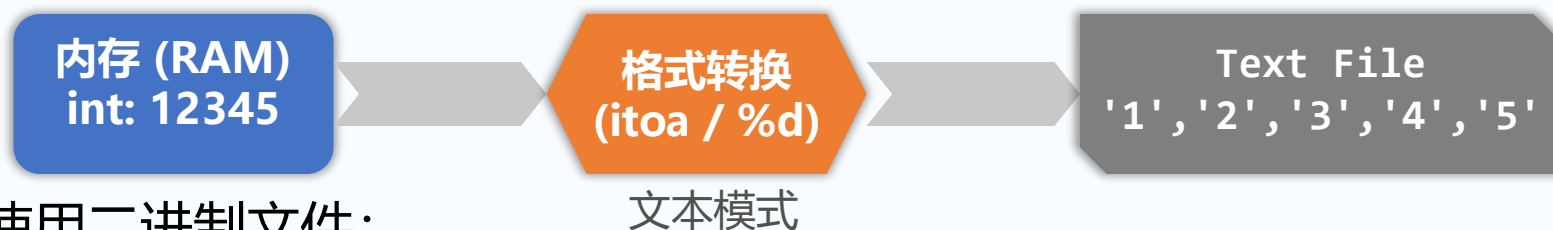
文本文件与二进制文件

❖ 存储整数 12345 的区别:

❖ 使用文本文件:

❧ 存储为5个字符 '1','2','3','4','5' (5字节)。

❧ 需经过 itoa 转换, 解析慢。



❖ 使用二进制文件:

❧ 存储为整数原码 0x3039 (4字节)。

❧ 内存直接映射, 速度极快。





序列化与反序列化

- ❖ 序列化 (Serialization): 将内存中复杂的数据结构 (对象、图、树) 转换为线性字节流的过程
 - ∞ 空间效率: 通常比文本更省空间 (浮点数尤甚)
 - ∞ 时间效率: 无须格式解析, 适合大数据吞吐
 - ∞ 应用场景: 数据持久化 (存盘)、网络传输 (Socket)、进程间通信
- ❖ 反序列化 (Deserialization): 将字节流还原为内存对象





数组的序列化

❖ 序列化：先写入数组的长度，然后写入整个数组的数据块

```
// 序列化数组：将数组保存到文件
// 格式：[元素个数 (int)] + [所有数据 ( [ ] )]
void serialize_array(const char *filename, int *array, int count)
{
    FILE* fp = fopen(filename, "wb");
    if (!fp) { perror("写入失败"); return; }

    // 1. 先写入数组长度 (Header)
    fwrite(&count, sizeof(int), 1, fp);

    // 2. 连续写入整个数组的数据块 (Data)
    fwrite(array, sizeof(int), count, fp);
    fclose(fp);
}
// 使用方法：serialize_array("arr.bin", a, 10);
```



数组的反序列化

❖ 反序列化：先读出数组长度并分配内存，然后读取整个数组

```
// 反序列化数组：从文件读取并创建新数组
// 返回值：指向动态分配数组的指针
int* deserialize_array(const char *filename, int *count) {
    FILE* fp = fopen(filename, "rb");
    if (!fp) { perror("读取失败"); return NULL; }
    // 1. 先读出数组长度
    fread(count, sizeof(int), 1, fp);
    // 2. 根据长度动态分配内存
    int* array = (int* )malloc(*count * sizeof(int));
    if (!array) { fclose(fp); return NULL; }
    // 3. 一次性将剩余数据读入内存
    fread(array, size, *count, fp); fclose(fp);
    return array;
}
// int *arr = deserialize_array("arr.bin", len);
```



简单结构体的序列化

- ❖ 对于不包含指针成员的简单结构体，由于结构体是连续存储，序列化和反序列化可通过直接内存拷贝完成

```
void serialize_struct(const char *filename, void *st, int size)
{
    FILE *fp = fopen(filename, "wb");
    if (fp) { // 直接将整个结构体的内存块写入文件
        fwrite(st, size, 1, fp); fclose(fp);
    }
}

void deserialize_struct(const char *filename, void *st, int size)
{
    FILE *fp = fopen(filename, "rb");
    if (fp) { // 直接将文件中的二进制流读入结构体变量
        fread(st, size, 1, fp); fclose(fp);
    }
}
```



简单结构体的序列化（例）

❖ 例：学生信息的序列化和反序列化

```
typedef struct {  
    int id;  
    char name[20];  
    float score;  
} Student;  
  
Student s1 = {1001, "Alice", 95.5f};  
serialize_struct("student.bin", &s1, sizeof(Student));  
printf("序列化成功：数据已保存到二进制文件。\\n");  
  
Student s2;  
deserialize_struct("student.bin", &s2, sizeof(Student));  
printf("反序列化成功：\\nID: %d, Name: %s, Score: %.1f\\n", s2.id,  
s2.name, s2.score);
```



结构体的紧凑对齐

- ❖ 编译器为了提高 CPU 读取内存的效率，通常会在结构体成员之间插入一些的填充字节，如：`struct { char c; int i; }` 大小是8字节，填充3字节
 - ⌘ 问题一：如果直接 `fwrite()` 这个结构体，垃圾填充字节也会被写入文件，会导致文件膨胀，浪费磁盘空间
 - ⌘ 问题二：如果把这 8 个字节写入文件，然后拷贝到另一台对齐规则不同的机器上读取，数据就会错位
- ❖ 解决方案：`#pragma pack(n)`
 - ⌘ 取 $n = 1$ ，采用紧凑模式，强制结构体按 1 字节对齐（即不留空隙）



结构体的紧凑对齐

❖ 不同对齐方式的内存差异

默认对齐 (Pack 4) ($n = 4$)



1B数据 + 3B填充 + 4B数据 = 8 Bytes

`#pragma pack(2)` ($n = 2$)



1B数据 + 1B填充 + 4B数据 = 6 Bytes

`#pragma pack(1)` ($n = 1$)



1B数据 + 0填充 + 4B数据 = 5 Bytes (紧凑)

```
struct {  
    char c;  
    double d;  
};
```

对齐模数 = $\min(\text{成员自身大小}, \text{pack系数})$

例如:

1. `pack(4)`:

char (1) -> 按 $\min(1, 4) = 1$ 对齐

double (8) -> 按 $\min(8, 4) = 4$ 对齐
(偏移量必须是4的倍数)

2. `pack(8)`:

double (8) -> 按 $\min(8, 8) = 8$ 对齐
(偏移量必须是8的倍数)



结构体的紧凑对齐

❖ 权衡：结构体对齐的优势与劣势

策略	优势	劣势
自然对齐 (Default)	速度快：CPU 访问内存效率最高（一次总线周期）	浪费空间：由于填充字节的存在，文件会变大，结构体不能直接用于网络传输
紧凑对齐 (Pack 1)	空间省：无垃圾数据，适合文件存储和网络协议	速度慢：CPU 可能需要两次读取才能拼凑出一个整数，强制使用指定字节对齐，可能会导致严重的性能下降



结构体的紧凑对齐

- ❖ 例：设置对齐系数中的作用域，只对用于序列化的结构体进行紧凑对齐

```
#pragma pack(push, 1) // 保存当前对齐状态，并设置新对齐为 1 字节
typedef struct {
    int id;
    char name[20];
    float score;
} PackedStudent; // 强制1字节对齐的紧凑表示，用于序列化
#pragma pack(pop) // 恢复之前的默认对齐状态

Student s1 = {1001, "Alice", 95.5f}; // 默认对齐，保证访问速度
PackedStudent ps1; // 手动将s1成员的值依次复制到ps1中
// 进行序列化
serialize_struct("student.bin", &ps1, sizeof(PackedStudent));

PackedStudent ps2; // 结构体紧凑对齐，用于反序列化
// 进行反序列化
deserialize_struct("student.bin", &ps2, sizeof(PackedStudent));
Student s2; // 手动将ps2成员的值依次复制到s2中
```



指针与深拷贝

- ❖ 当结构体包含指针成员时，直接 `fwrite()` 的浅拷贝方式，只保存了一个无效的指针地址 (如 `0x7FFF001`) 到文件中
- ❖ 下次读取文件时，该地址已不可用，不能得到信息，还会导致程序崩溃，如：

```
typedef struct {  
    char *name;  
    int id;  
} Student;
```

```
Student s = { 101, "Bob" };
```

```
// 典型错误写法：直接写入指针（直接 fwrite 整个结构体）
```

```
// 后果：写入文件的仅仅字符串在当前内存中的地址，而非字符串本身！
```

```
fwrite(&s, sizeof(Student), 1, fp);
```



指针与深拷贝

- ❖ 正确的做法是进行**深拷贝**，把指针指向的实际内容写入文件

```
// 标准正确写法：序列化协议
// 先存 name 的长度，再存 name 的内容，最后存 id
// 1. 获取字符串长度
// 这里我们约定：先存长度(int)，再存内容
int name_len = strlen(s.name);
// 2. 写入名字长度
fwrite(&name_len, sizeof(int), 1, fp);
// 3. 写入名字的实际内容（深拷贝）
fwrite(s.name, sizeof(char), name_len, fp);
// 4. 写入 ID
fwrite(&s.id, sizeof(int), 1, fp);
```

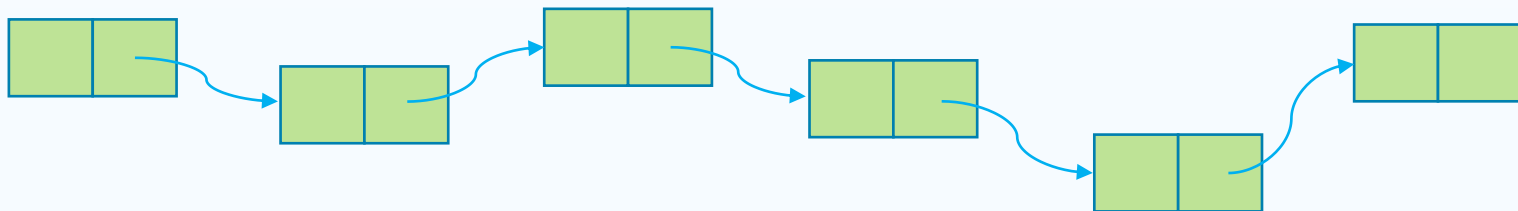
```
typedef struct {
    char *name;
    int id;
} Student;
```

```
// 反序列化时，根据上述序列化协议
// 先读取name的长度，再读取name的内容，最后读取id值
```



链表的序列化与反序列化

- ❖ 链表的每个节点由结构体表示，不仅包含数据域，同时包含指向下一个节点的指针域，如：`struct Node { int data; struct Node *next; }`
- ❖ 序列化时，不能只 `fwrite()` 头节点，必须遍历整个链表，逐个写入：
 1. 写入链表长度 N
 2. 循环 N 次，写入每个节点的数据
- ❖ 反序列化时，先读 N，再循环 N 次 `malloc()` 节点并重新连接





链表的序列化（例）

// 深度序列化：将链表完整写入二进制文件

```
FILE *fp = fopen(filename, "wb");
```

// 1. 先统计节点个数

```
int count = 0;
```

```
Node *cur = head;
```

```
while (cur) {  
    count++; cur = cur->next;  
}
```

// 2. 写入节点个数

```
fwrite(&count, sizeof(int), 1, fp);
```

// 3. 遍历链表，依次写入每个节点的数据

```
cur = head;
```

```
for (int i = 0; i < count; i++) {  
    fwrite(&cur->data, sizeof(int), 1, fp); cur = cur->next;  
}
```

```
fclose(fp);
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```



链表的反序列化（例）

// 深度反序列化：从二进制文件重建链表

```
FILE *fp = fopen(filename, "rb");
```

// 1. 读取节点个数

```
int count = 0;
```

```
fread(&count, sizeof(int), 1, fp);
```

// 2. 依次读取每个节点的数据，并重建链表

```
Node *head = NULL, *tail = NULL;
```

```
for (int i = 0; i < count; i++) {
```

```
    int data; fread(&data, sizeof(int), 1, fp);
```

```
    Node *new_node = create_node(data);
```

// 将新节点链接到链表尾部

```
    if (head == NULL) { head = tail = new_node; }
```

```
    else { tail->next = new_node; tail = new_node; }
```

```
}
```

```
fclose(fp);
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```




魔数校验

- ❖ 魔数 (Magic Number) 是位于文件开头的几个固定字节 (比如 0x46494C45, 对应 ASCII 的 “FILE”), 用来快速验证该文件的格式
- ❖ 魔数用于验证输入文件的合法性, 防止程序因处理不符合预期格式的文件而崩溃或被恶意利用 (例如病毒程序伪装为图片, 欺骗用户点击)
- ❖ 例如, 一些常用的文件格式都有魔数:
 - ☞ PDF文件: 开头固定是 %PDF (0x25 50 44 46)
 - ☞ Java Class文件: 开头永远是 0xCAFEBAFE (咖啡宝贝)
 - ☞ PNG 图片: 开头是 0x89 50 4E 47
 - ☞ Windows 可执行文件 (EXE): 开头是 MZ (0x4D 5A)
- ❖ 序列化时, 在二进制文件开头写入魔数, 标记文件格式 (序列化协议)
- ❖ 反序列化时, 在正式解析二进制文件内容前, 先验证魔数并确定文件格式, 避免后续代码因格式错误而崩溃



魔数校验（例一）

- ❖ 例：不依赖文件后缀名（.txt也可改为.png），通过读取文件的前几个字节（魔数）来判断该图片是否是PNG格式

```
// PNG 文件的魔数
// 8字节，十六进制为 89 50 4E 47 0D 0A 1A 0A
static const uint8_t png_magic[8] = {0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A};
```

```
FILE *fp = fopen(filename, "rb");
uint8_t header[8];
size_t bytes_read = fread(header, 1, 8, fp);
fclose(fp);
```

```
// 比较前8字节
int is_png = memcmp(header, png_magic, 8) == 0;
```

Hex十六进制格式							
89	50	4e	47	0D	0A	1A	0A
魔术字	P	N	G	DOS-Unix系统的换行符CRLF		DOS系统的换行符	Unix-DOS系统的换行符LF



魔数校验 (例二)

```
#define MY_MAGIC_NUMBER 0x55535443 // "USTC" 的十六进制
#define CURRENT_VERSION 1
typedef struct {
    uint32_t magic; // 魔数
    uint16_t version; // 版本号
} FileHeader;

void save_data(const char* filename, const Student *ps) {
    FILE* fp = fopen(filename, "wb");
    if (!fp) return;
    FileHeader header = {
        .magic = MY_MAGIC_NUMBER,
        .version = CURRENT_VERSION
    };
    // 1. 写入文件头, 包含魔数与版本号
    fwrite(&header, sizeof(FileHeader), 1, fp);
    ... // 2. 根据序列化协议, 写入数据
    fclose(fp);
}
```



魔数校验 (例二续)

```
void load_data(const char* filename, Student *ps) {  
    FILE* fp = fopen(filename, "rb");  
    if (!fp) return -1;  
    FileHeader header;  
    // 1. 读取头部  
    if (fread(&header, sizeof(FileHeader), 1, fp) != 1) {  
        fclose(fp); return -2; //错误: 无法读取文件头  
    }  
    // 2. 核心步骤: 魔数检查  
    if (header.magic != MY_MAGIC_NUMBER) {  
        fclose(fp); return -3; //校验失败: 这可能不是一个有效的USTC存档文件  
    }  
    // 3. 可选: 版本检查  
    if (header.version > CURRENT_VERSION) {  
        fclose(fp); return -4; //校验失败: 存档版本过高, 请更新程序  
    }  
    ... // 4. 校验通过, 提取数据  
    fclose(fp);  
    return 0; // 成功  
}
```



本章内容概述

- ❖ 文件操作与数据持久化
- ❖ 二进制文件与序列化
- ❖ 文件的定位与随机访问
- ❖ 文件的错误处理



文件读写位置指针

❖ 文件指针与文件读写位置指针

- ☞ 指示文件的当前读写位置，在打开或创建文件时由系统自动创建
- ☞ 每次对文件进行读写操作后该指针自动更新位置
- ☞ 该指针是一个整数，表示当前位置（字节为单位）

❖ 文件读写位置指针的定位

- ☞ 文件打开时，该指针位于文件开始或末尾（模式a），顺序读写文件时，该指针自动移动
- ☞ 随机读写（不是从头顺序读写）时，需要先将读写位置指针移动到特定的位置，称为**文件定位**



文件定位

❖ 位置指针定位到指定位置（随机读写）

`int fseek(FILE *stream, long offset, int origin);`

☞ origin为位置指针的**移动参考点**:

❖ 文件头 `SEEK_SET (0)`

❖ 当前位置 `SEEK_CUR (1)`

❖ 文件尾 `SEEK_END (2)`

```
for(count=1L; count<=size; count++) {  
    fseek(fp, -count, SEEK_END);  
    ch = fgetc(fp); // 逆向输出文件的所有字节  
}
```

☞ offset 为从参考点的**位移量**(字节数、长整型)，可以为正值（向文件末尾移动）、负值（向文件开始处移动）或 0（保持不动）

❖ 位置指针（重新）定位到开头

`void rewind(FILE *stream);`

☞ 基本等价于 `fseek(stream, 0, SEEK_SET)`，用于将文件指针设置到文件的起始位置，区别是会清除当前文件的错误指示器



文件定位

❖ fseek() 函数的调用:

- ❧ `fseek(fp, 100L, SEEK_SET);` // 从文件头向后（文件尾）移动100个字节
- ❧ `fseek(fp, -10L, SEEK_CUR);` // 从当前位置向前（文件头）移动10个字节
- ❧ `fseek(fp, -20L, SEEK_END);` // 从文件尾向前（文件头）移动20个字节

❖ 如以读写模式打开文件（如 r+、w+、a+）

- ❧ **不改变模式**：仅移动位置指针，不论当前处于读模式还是写模式。
- ❧ **读写切换规范**：如果在读操作后紧接着进行写操作（或反之），必须在中间插入一个 `fseek` 或 `fflush` 调用，以重置流的读写属性。
- ❧ **适用场景**：即使 `fseek` 到当前位置（例如 `fseek(fp, 0, SEEK_CUR)`），它也能起到刷新缓冲、允许从读转写或从写转读的作用。

❖ 注意：fseek()最好只用来操作二进制文件，不要用来读取文本文件。因为文本文件的字符有不同的编码，某个位置的准确字节位置不容易确定



文件定位

❖ 使用 `ftell()` 函数获取文件位置指针的当前位置

`long ftell(FILE *stream);`

- ☞ 调用成功则返回当前文件指针的位置（以字节为单位），即文件指针相对于文件开头的字节位置；失败时则返回-1L
- ☞ `ftell()` 可以跟 `fseek()` 配合使用，先记录内部指针的位置，一系列操作过后，再用 `fseek()` 返回原来的位置

```
long file_pos = ftell(fp); // 记录当前位置
// 一系列文件操作之后，返回之前的位置
fseek(fp, file_pos, SEEK_SET);
```

- ☞ 还可以先将定位到文件结尾，然后得到文件的总字节数

```
fseek(fp, 0L, SEEK_END); // 将指示器定位到文件结尾
long size = ftell(fp); // 得到文件开始处到结尾的字节数，即文件的大小
```



文件定位

- ❖ `fseek()` 和 `ftell()` 有一个潜在的问题，那就是它们都把文件大小限制在 `long int` 类型能表示的范围内（在32位计算机上，`long int` 的长度为4个字节，能够表示的范围最大只有 4GB）
- ❖ 随着存储设备的容量迅猛增长，文件也越来越大，往往会超出这个范围。因此C语言新增了两个处理大文件的新定位函数：`fgetpos()` 和 `fsetpos()`

`int fgetpos(FILE* stream, fpos_t* pos);`

`int fsetpos(FILE* stream, const fpos_t* pos);`

```
fpos_t file_pos;  
fgetpos(fp, &file_pos); // 记录当前位置，类似于file_pos = ftell(fp)  
  
// 一系列文件操作之后，返回之前的位置  
fsetpos(fp, &file_pos); // 作用类似于fseek(fp, file_pos, SEEK_SET)
```



文件定位（例一）

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    FILE *fp;
    char ch;
    if ((fp = fopen("test4.txt", "r+")) == NULL) {
        printf("can't open this file.\n");
        exit();
    }
    ch = fgetc(fp);
    while (!feof(fp)) {
        if (ch >= 'A' && ch <= 'Z') {
            ch += 32;
            fseek(fp, -1, SEEK_CUR);
            fputc(ch, fp);
            fseek(fp, 0, SEEK_CUR);
        }
        ch = fgetc(fp);
    }
    fclose(fp);
}
```

文件以读写的方式打开，r+ 代表第一个操作是读

读写位置指针回移，重置流的读写属性，准备切换为写

读写位置指针不动，将缓冲区内容写入文件，准备切换为读



文件定位（例二）

❖ 例：实现学生数据库，不读前2个，直接读第3个学生信息

∞ 原理：利用结构体大小定长特性计算偏移量，跳过前两个数据块

∞ 公式： $\text{Offset} = \text{Index} * \text{sizeof}(\text{Student})$

```
typedef struct {
    int id; char name[1024]; float scores[256];
} Student;
FILE* fp = fopen(filename, "rb");
// 跳过前两个: Offset = Index * sizeof(Struct)
long offset = index * sizeof(Student);

// 1. 移动指针 (跳过前 index 个)
fseek(fp, offset, SEEK_SET);
// 2. 读取数据
Student stu;
fread(&stu, sizeof(Student), 1, fp);
```



文件定位（例三）

❖ 例：实现数据的原地修改，将第5个数据的数值从10改为 99

❧ 模式必须是 “rb+”（读写模式），否则文件会被清空

❧ 写入后，文件指针会自动后移，无需手动调整

```
// 如果用 "wb", 文件会被清空；如果用 "rb", 无法写入。  
FILE *fp = fopen(FILENAME, "rb+");  
int newValue = 99;  
long offset = 4 * sizeof(int);  
  
// 移动文件指针 (fseek)  
fseek(fp, offset, SEEK_SET);  
// 覆盖写入 (fwrite)  
fwrite(&newValue, sizeof(int), 1, fp);  
// 关闭文件以保存修改  
fclose(fp);
```



文件定位（例四）

❖ 例：大文件断点续传与分块处理

```
// 举例：断点续传
// 获取本地目标文件已下载的大小
FILE *fp = fopen(filename_dest, "rb");
fseek(fp, 0, SEEK_END); long size = ftell(fp); fclose(fp);

FILE *src = fopen(filename_src, "rb");
fseek(src, size, SEEK_SET); // 远程源文件：移动到下载断点

// 本地目标文件：追加模式 ("ab") -> 写入指针自动在末尾
FILE *dest = fopen(filename_dest, "ab");

// 大文件分块处理：分块拷贝文件
char buf[BUFFER_SIZE]; // 4KB 缓冲区
while (n = fread(buf, 1, BUFFER_SIZE, src) > 0) {
    fwrite(buf, 1, n, dest);
}
```



文件记录删除

❖ 文本文件：必须重写整个文件，去掉那一行（重写策略）

- ❧ 文本文件被视为一个连续的字节序列，缺乏内部的结构化索引；要在中间删除一行，本质上是在进行字符数组（字符串）的删除操作；由于文件存储在磁盘上，无法像在内存中那样轻易移动后续数据
- ❧ 因此必须执行复制交换（Copy-and-Swap）策略：读取源文件，过滤掉目标行，写入临时文件，最后替换原子操作；该操作伴随着大量的磁盘 I/O 开销（读写整个文件），这在大数据量场景下是不可接受的性能瓶颈

❖ 二进制文件：通常使用“惰性删除”（Lazy Delete）

- ❧ 在定长的二进制记录中，对数据块拥有随机访问的能力，可以通过修改结构体中的状态位，将其标记为‘无效’，因此也称为标记删除法（Flagging）
- ❧ 此时数据物理上依然存在，但逻辑上已被移除；该操作仅产生极小的 I/O（单次写入），是现代数据库引擎和文件系统底层的通用做法



文件记录删除（例）

- ❖ 结构体增加字段： `int isDeleted;`
- ❖ 删除时： `fwrite()` 将 `isDeleted` 设为1
- ❖ 读取时：如果 `isDeleted` 为 1 则跳过
 - ☞ 优点：删除的速度极快 ($O(1)$)
 - ☞ 副作用：虽然很快，但占用的空间并没有释放。如果删了很多数据，文件会变得很虚大（有效数据很少，文件却很大）
- ❖ 撤销时（Undo）：只需要写一个函数把 `isDeleted` 改回 0

```
// 定义结构体包含状态标记位
typedef struct {
    ...
    // 0 = 有效 (Active)
    // 1 = 已删除 (Deleted)
    int isDeleted;
} Record;
```




索引文件

- ❖ 如果学生ID不是连续的（如 202301, 202399...）或者结构体大小不一致，无法用数学公式计算 Offset，因此无法实现随机访问
- ❖ 解决方案：建立一个索引表 (Index Table)
 - ❧ 主数据文件 (Data.bin):
 - ❖ 乱序存放学生数据，不定长。
 - ❖ 模拟真实环境中数据追加写入的情况。
 - ❧ 索引文件 (Index.bin): 只存 {ID, Offset}，定长，按ID排序。
 - ❧ 查询操作：
 - ❖ 先在 Index.bin 二分查找 ID，拿到 Offset;
 - ❖ 在 Data.bin 中直接跳到指定位置读取数据



索引文件（例）

❖ 例：主数据文件与索引文件

// 主数据：学生详情（体积较大，包含非关键信息等）

```
typedef struct {  
    int id;  
    char name[32];  
    char bio[128]; // 模拟大数据字段  
    int score;  
} Student;
```

// 索引项：定长、轻量（仅 8-12 字节）

```
typedef struct {  
    int id; // Key  
    long offset; // Value: 在 data.bin 中的字节偏移量  
} IndexItem;
```



索引文件（例续）

❖ 存储程序：构建数据与索引，并分别保存为文件

```
void save_records(Student students[]) {  
    FILE *f_data = fopen("data.bin", "wb");  
    FILE *f_idx = fopen("index.bin", "wb");  
    if (!f_data || !f_idx) return;  
  
    for (int i = 0; i < 3; i++) {  
        // 1. 获取当前数据文件写入位置的偏移量  
        long current_offset = ftell(f_data);  
        // 2. 写入主数据  
        fwrite(&students[i], sizeof(Student), 1, f_data);  
        // 3. 构建索引项并写入索引文件  
        IndexItem item = {students[i].id, current_offset};  
        fwrite(&item, sizeof(IndexItem), 1, f_idx);  
    }  
    fclose(f_data); fclose(f_idx);  
} // 数据与索引存储完毕
```



索引文件（例续）

❖ 读取程序：通过索引随机访问

```
void find_student_by_id(int target_id) {
    FILE *f_idx = fopen("index.bin", "rb");
    FILE *f_data = fopen("data.bin", "rb");
    if (!f_idx || !f_data) return;
    IndexItem item; int found = 0;
    while (fread(&item, sizeof(IndexItem), 1, f_idx)) {
        // 1. 在索引文件中查找 ID
        if (item.id == target_id) {
            // 2. 找到索引后，利用 offset 直接定位数据文件
            fseek(f_data, item.offset, SEEK_SET);
            Student s; fread(&s, sizeof(Student), 1, f_data);
            printf("查询成功! \nID: %d\n姓名: %s\n简介: %s\n分数: %d\n", s.id, s.name, s.bio, s.score); found = 1; break;
        }
    }
    if (!found) printf("未找到 ID 为 %d 的学生记录.\n", target_id);
    fclose(f_idx); fclose(f_data);
}
```



本章内容概述

- ❖ 文件操作与数据持久化
- ❖ 二进制文件与序列化
- ❖ 文件的定位与随机访问
- ❖ 文件的错误处理



文件操作错误

- ❖ 常见错误：文件被占用、磁盘已满、权限不足等。
- ❖ 打开文件时，检查 `fopen()` 返回值
 - ⌘ 如果返回 `NULL`，说明文件打开失败
- ❖ 读写文件过程中，使用状态检查函数
 - ⌘ `feof(fp)`: 检查是否自然读到了文件末尾
 - ⌘ `ferror(fp)`: 检查最近一次操作是否发生了读写错误

```
// 循环结束有两种可能：读完了(EOF) 或者 出错了(Error)
if (ferror(fp)) {
    // 硬件/系统错误检测 (如磁盘坏道、设备被拔出)
    fprintf(stderr, "读取过程中发生 I/O 错误\n");
} else if (feof(fp)) {
    printf("已安全到达文件末尾 (EOF)。 \n"); // 自然结束检测
}
```



错误诊断函数

❖ 使用全局变量 `errno` (`#include <errno.h>`)

- ❧ 操作系统维护的一个全局整数，记录了最近一次系统调 用的错误码
- ❧ 例如：2 (`ENOENT` - 无此文件) ， 13 (`EACCES` - 无权限)
- ❧ 使用 `strerror(errno)` 函数可以将错误码转换为文字描述

❖ 使用 `perror()` 函数输出错误信息

`void perror(const char *str);`

- ❧ 参数：str 为自定义的错误消息前缀，通常是描述错误来源的字符串。它会与 `errno` 中的错误信息一起输出
- ❧ 输出：输出的错误信息包括自定义前缀和 `errno` 对应的系统错误描述，二者用冒号分隔。通常输出到标准错误流 (`stderr`)
- ❧ 效果相当于 `fprintf(stderr, "%s: %s" , str, strerror(errno));`



错误诊断函数（例）

```
#include <errno.h> // 必须包含：定义了 errno 全局变量
#include <string.h> // 必须包含：用于 strerror

// errno 会在函数调用前被系统自动清零或在错误时设置
FILE *fp = fopen(filename, "r");

// 防御性检查：指针是否有效
if (fp == NULL) {
    // 使用 perror 输出标准错误信息
    perror("无法打开文件 (perror)");
    // 如果文件没找到，输出：无法打开文件 (perror) : No such file or directory

    // 或者手动使用 errno 获取错误码（高级用法）
    fprintf(stderr, "错误码： %d， 描述： %s\n", errno, strerror(errno));

    // 遇到打开失败，必须终止后续文件操作
    return;
}
```




结束检查函数

❖ 使用 `feof()` 函数判断文件结束

`int feof(FILE *stream);`

- 文件读写位置指针移过了文件尾，返回非0值；否则返回0值
- 注意：`feof()` 只有在读取失败后才会变真，而不是读取前；所以即便是文件为空，也必须先读，判断结果，再处理

```
// 典型错误写法
char buf[100];
while (!feof(fp))
{
    // 假设这是最后一次循环：
    // 文件已读完，fgets失败，buf内容不变
    fgets(buf, 100, fp);
    // 错误点：这里打印了旧的 buf
    // 导致最后一行重复输出
    printf("%s", buf);
}
```

```
// 标准正确写法
char buf[100];
// 核心：读取的同时判断是否成功
// 只有 fgets 返回非 NULL 才进入循环
while (fgets(buf, 100, fp) != NULL)
{
    // 或者读取后用feof进行判断
    if (feof(fp)) break;
    // 读到了新数据
    printf("%s", buf);
}
```



错误检查函数

❖ 使用 `ferror()` 函数获取输入输出文件的错误

`int ferror(FILE *stream);`

- ❧ 若上一次读写函数调用出错，则返回非0值；否则返回0值
- ❧ 调用读写函数后，可以根据读写函数返回值判断是否出错
- ❧ 此外，系统会自动维护一个可被 `ferror()` 函数返回值，标记是否出错
- ❧ 可在调用输入输出函数后，立即调用 `ferror()` 函数，获取错误信息
- ❧ 执行 `fopen()` 后，`ferror()` 的初始值自动置 0

❖ 使用 `clearerr` 函数清除文件错误标志

`void clearerr(FILE *stream);`

- ❧ 清除文件错误标志 `ferror()` 和文件结束标志 `feof()`
- ❧ 只要读写发生错误，错误标志就会一直存在，直到对同一文件调用 `clearerr()` 函数、`rewind()` 函数、或任何其他一个读写函数



错误检查函数 (例)

```
#include <stdio.h>
int main() {
    FILE *file = fopen("example.txt", "r");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
    char buffer[100];
    if (fgets(buffer, sizeof(buffer), file) == NULL) {
        if (ferror(file)) {
            perror("Error reading file");
            fclose(file);
            return 1;
        }
        clearerr(file);
        puts(feof(file) ? "EOF indicator set"
              : "EOF indicator cleared");
        fclose(file);
        return 0;
    }
}
```

在尝试读取文件时，如果 fgets 返回 NULL，使用 ferror 检查是否发生了错误。如果 ferror 返回非零值，则调用 perror 输出错误信息。

同时清除文件错误标志 ferror() 和文件结束标志 feof()



缓冲区相关错误

- ❖ 由于存在缓冲机制，`fprintf()/fwrite()` 是先写到内存缓冲区，满了才写入磁盘，程序崩溃可能导致最后写入的数据丢失
- ❖ 使用 `fflush(fp)` 函数强制刷新缓冲区，将缓冲区数据立即写入磁盘，适用于写日志、交易记录等长时间运行的任务
 - ⌘ 正常的程序退出（`return 0` 或 `exit(0)`）时，C 标准库会自动调用 `fclose()`，而 `fclose()` 内部会自动执行 `fflush()`
 - ⌘ 注意：频繁调用 `fflush(fp)` 写入磁盘会影响系统性能

```
fprintf(fp, "This is SAFE DATA protected by fflush.");
```

```
// 如果此时断电，上述数据可能还在内存里！
```

```
fflush(fp); // 强制存入硬盘，确保安全
```

```
_exit(1); // 该系统调用会直接杀死进程，不给 C 标准库清理缓冲区的机会  
// 此处模拟了计算机断电、蓝屏等意外情况
```



文件缓冲区调整

❖ 文件缓冲区的核心思想是：与其频繁地读写磁盘，不如先将数据存在内存中，等到缓冲区满了或者达到特定条件时再统一处理

❖ 在打开文件后、执行输入输出操作前，调用 `setvbuf` 函数控制缓冲区

`int setvbuf(FILE *stream, char *buf, int mode, size_t size);`

∞ `stream` 是指向文件流的指针，`buf` 是用户自定义的缓冲区指针（如果传 `NULL`，库会自动分配），`size` 是缓冲区的大小（字节数）

∞ `mode` 可以有三种模式：

模式常量	名称	行为描述	典型应用
<code>_IOFBF</code>	全缓冲	缓冲区满时才进行实际写操作	读写大型二进制文件
<code>_IOLBF</code>	行缓冲	遇到换行符 <code>\n</code> 或缓冲区满时写操作	终端显示（ <code>stdout</code> 默认模式）
<code>_IONBF</code>	无缓冲	数据立即写入，不经过中间缓存	错误日志（ <code>stderr</code> 默认模式）



文件缓冲区调整 (例)

❖ 例：禁用缓冲区（实时日志输出）

```
// 设置为无缓冲模式
// 即使没有 \n, 每次 fprintf 都会立即触发系统写操作
setvbuf(fp, NULL, _IONBF, 0);
for (int i = 0; i < 5; i++) {
    fprintf(fp, "正在执行步骤 %d...", i);
    // 这里不需要 fflush, 内容也会立即进入文件
}
```

❖ 例：自定义超大缓冲区（提高大数据读写性能）

```
// 手动分配缓冲区
char *my_buffer = (char *)malloc(BUF_SIZE);
// 设置为全缓冲, 并指定 1MB 大小
if (setvbuf(fp, my_buffer, _IOFBF, BUF_SIZE) != 0) {
    perror("设置缓冲区失败");
}
// 执行大量写入...
// 数据会积攒到 1MB 后才真正写入硬盘一次
```



资源泄露错误

- ❖ 文件句柄是有限资源（涉及文件信息和缓冲区的维护），操作系统限制单进程打开文件数（通常为 512 或 1024）
- ❖ 常见 `fopen()` 的错误
 - ☞ 在循环中 `fopen()` 却忘记 `fclose()`
 - ☞ 函数中间遇到错误提前 `return`，跳过了 `fclose()`

```
for (int i=0; i<5000; i++) {  
    FILE *fp = fopen("temp.txt", "r");  
    // 忘记 fclose(fp)  
}  
// 运行不久后, fopen 将一直返回 NULL
```



系统临时文件

- ❖ 有时需要使用文件暂存临时数据，用完后需要手动删除，否则大量生成的文件将可能迅速塞满磁盘
- ❖ 解决方案：使用函数 `tmpfile()` 创建临时文件
- ❖ 系统临时文件的特点：
 - ∞ 在系统临时目录创建文件。
 - ∞ 程序结束或 `fclose()` 时自动删除，不留垃圾。

```
#include <stdio.h>

FILE *temp = tmpfile();
fprintf(temp, "Temporary Data");
rewind(temp);
// ... 读取使用 ...
fclose(temp); // 文件自动消失
```




系统临时文件

❖ 临时文件存储在系统的临时目录

☞ 在 Windows 下：

- ❖ 通常取决于环境变量 `%TEMP%` 或 `%TMP%`。
- ❖ 默认路径：`C:\Users\<UserName>\AppData\Local\Temp`

☞ 在 Linux / macOS 系统下：

- ❖ 默认路径：`/tmp`
- ❖ 备用路径：`/var/tmp`（具体取决于 `<stdio.h>` 中定义的 `P_tmpdir` 宏）

❖ 匿名性：文件名通常是系统自动生成的随机乱码（如 `tmp78A.tmp`）

❖ 立即“删除”机制（Unix/Linux 特性）：程序一旦崩溃或结束，文件自动消失，不会残留垃圾在磁盘上



文件的删除与重命名

❖ 删除指定文件（相当于 Linux 中的 `rm` 命令）

```
int remove(const char* filename);
```

- ❧ 接受文件名作为参数。如果删除成功，返回0，否则返回非零值
- ❧ 注意：删除文件必须是在文件关闭的状态下。如果是用 `fopen()` 打开的文件，必须先用 `fclose()` 关闭后再删除

❖ 重命名文件（相当于 Linux 中的 `mv` 命令）

```
int rename(const char* old_filename, const char* new_filename);
```

- ❧ 它接受两个参数，第一个参数是现在的文件名，第二个参数是新的文件名。如果改名成功，返回0，否则返回非零值
- ❧ 注意：改名后的文件不能与现有文件同名；另外，如果要改名的文件已经打开了，必须先关闭，然后再改名，对打开的文件进行改名会失败

```
// 通过重命名来移动文件
```

```
rename("/tmp/evidence.txt", "/home/user/nothing.txt");
```



谢谢!



中国科学技术大学

University of Science and Technology of China